

Exercise 2

SQL Aggregate Functions & Operators

1. List all distinct department in students table

```
SELECT DISTINCT department  
FROM student;
```

department

IT

HR

Finance

2. Average age per department

```
SELECT department, AVG(age) AS avg-age  
FROM student  
GROUP BY department;
```

department	avg-age
IT	21.5
HR	21
Finance	22

3. Show department with more than one students

```
SELECT department, (COUNT(*)) AS student-count  
FROM student  
GROUP BY department  
HAVING (COUNT(*)) > 1
```

department	student-count
IT	2

4. Get all students whose age is between 21 and 23

```
SELECT student_id, name, age  
FROM students  
WHERE age BETWEEN 21 AND 23;
```

student_id	name	age
2	Bob	22
3	Charlie	21
4	Diana	23
5	Eve	22

5. List all student in the IT or HR department who are older than 21.

SELECT student_id, name, age, department
FROM students

WHERE department IN ('IT', 'HR')

AND age > 21;

student_id	name	age	department
2	Bob	22	IT
4	Diana	23	HR

6. Show total credits per department, only for departments with more than 5 total credits.

SELECT department, SUM(credits) AS total_credits
FROM courses

GROUP BY department

HAVING SUM(credits) > 5;

department	total_credits
IT	11

7. List all course that do not have 4 credits

SELECT course_id, course_name, department, credits

FROM courses

WHERE credits != 4;

Course_id	course_name	department	credits
101	SQL basis	IT	3
104	Excel	Finance	2
105	Statistics	HR	2

8. Show the top 3 course by credit in descending order

```
SELECT course_id, course_name, department, credit
FROM courses
ORDER BY credit DESC
LIMIT 3;
```

course_id	course_name	department	credit
102	Python	IT	4
103	Data Science	IT	4
101	SQL Basis	IT	3

9. Get maximum, minimum, and average grade across all enrollment

```
SELECT MAX(grade) AS max_grade,
       MIN(grade) AS min_grade,
       AVG(grade) AS avg_grade
FROM enrollments;
```

max_grade	min_grade	avg_grade
90	78	84.33

10. Count how many enrollment exist per course

```
SELECT course_id, COUNT(*) AS enrollment_count
FROM enrollments
GROUP BY course_id
```

course_id	enrollment_count
101	1
102	1
103	1
104	1
105	1

11. Find total Salary and total bonus per department

```
SELECT department,
      SUM(Salary) AS total_salary,
      SUM(bonus) AS total_bonus
FROM salaries
GROUP BY department;
```

department	total_salary	total_bonus
IT	122000	10500
HR	109000	8500
Finance	70000	6000

12. Show department where average salary is above 55000

```
SELECT department,
      AVG(Salary) AS avg_salary
FROM salaries
GROUP BY department
HAVING AVG(Salary) > 55000;
```

department	avg_salary
IT	61000
Finance	70000

13. List employees whose salary plus bonus is greater than 60000

```
SELECT employee_id, name, salary, bonus, SUM(salary + bonus) AS total_compensation
FROM salaries
WHERE (salary + bonus) > 60000;
```

employee_id	name	salary	bonus	total_compensation
1	Tom	60000	5000	65000
3	Spike	70000	6000	76000
4	Tyke	62000	5500	67500

14 Show total and average budget per department only include department with average budget above 70000.

SELECT department,
SUM(budget) AS total_budget
AVG(budget) AS avg_budget

FROM projects

GROUP BY department

HAVING (avg_budget) > 70000;

department	total_budget	avg_budget
IT	270000	135000
finance	80000	80000

15. List all projects with budgets between 50000 and 120000, excluding the marketing department

SELECT project_id,
project_name,
department,
budget

FROM projects

WHERE budget BETWEEN 50000 AND 120000

AND department <> 'Marketing';

Project id	project name	department	budget
1	AI App	IT	120000
2	Program system	finance	80000
5	HR portal	HR	50000